



TOPSTECHNOLOGIES

Training | Outsourcing | Placement | Study Abroad

Module-8

JavaScript Essentials And Advanced



An Introduction to JavaScript

- **What is JavaScript?**
- JavaScript was initially created to “make web pages alive”.
- The programs in this language are called scripts. They can be written right in a web page’s HTML and run automatically as the page loads.
- Scripts are provided and executed as plain text. They don’t need special preparation or compilation to run.

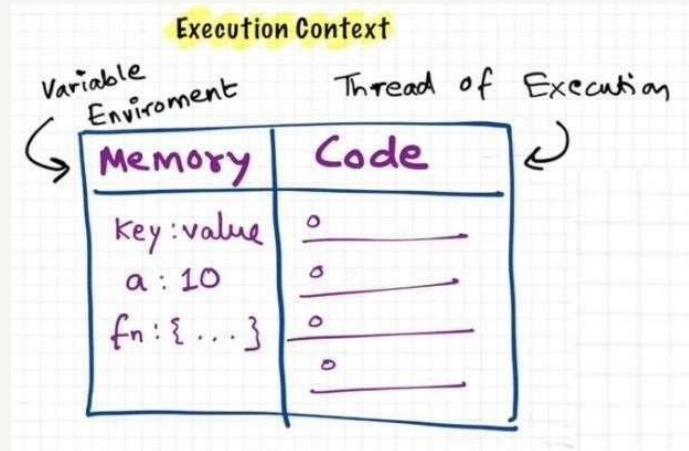
An Introduction to JavaScript

In this aspect, JavaScript is very different from another language called Java.

- JavaScript ("JS" for short) is a full-fledged dynamic programming language that, when applied to an HTML document, can provide dynamic interactivity on websites. It was invented by Brendan Eich, co-founder of the Mozilla project, the Mozilla Foundation, and the Mozilla Corporation.

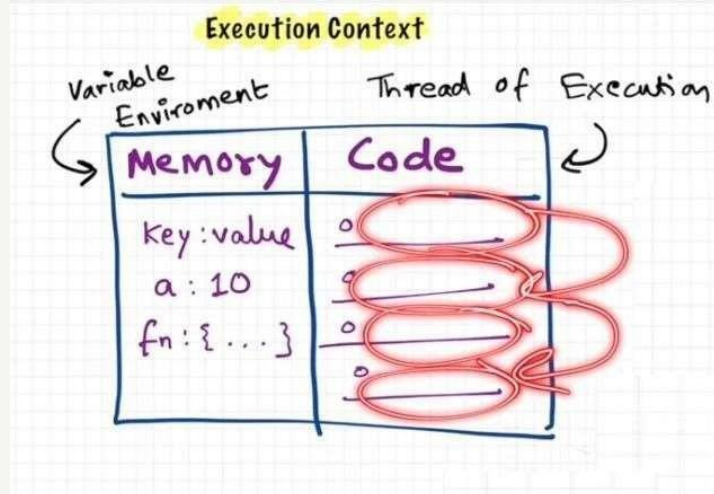
An Introduction to JavaScript

Everything in JavaScript Happens inside an execution context



An Introduction to JavaScript

JavaScript is asynchronous Single-threaded Language



Execution

Memory	Code				
$n = 2$ $\text{square}:\{\dots\}$ $\text{square2}:$ undefined $\text{square4}:$ undefined	<table><tr><th>Memory</th><th>Code</th></tr><tr><td>$\text{num};$ $\text{ans};$ undefined</td><td></td></tr></table>	Memory	Code	$\text{num};$ $\text{ans};$ undefined	
Memory	Code				
$\text{num};$ $\text{ans};$ undefined					

```
1 var n = 2;  
2 function square (num) {  
3     var ans = num * num;  
4     return ans;  
5 }  
6 var square2 = square(n);  
7 var square4 = square(4);
```

Why is it called JavaScript?

- When JavaScript was created, it initially had another name: “LiveScript”. But Java was very popular at that time, so it was decided that positioning a new language as a “younger brother” of Java would help.
- But as it evolved, JavaScript became a fully independent language with its own specification called
- **ECMAScript**(European Computer Manufacturers Association.), and now it has no relation to Java at all.

- Today, JavaScript can execute not only in the browser, but also on the server, or actually on any device that has a special program called the JavaScript engine.
- The browser has an embedded engine sometimes called a “JavaScript virtual machine”.
- Different engines have different “codenames”. For example:
 - V8 – in Chrome and Opera.
 - SpiderMonkey – in Firefox.
- ...There are other codenames like “Trident” and “Chakra” for different versions of IE, “ChakraCore” for Microsoft Edge, “Nitro” and “SquirrelFish” for Safari, etc.
- The terms above are good to remember because they are used in developer articles on the internet. We’ll use them too. For instance, if “a feature X is supported by V8”, then it probably works in Chrome and Opera.

THE EVOLUTION OF JAVASCRIPT

- JavaScript is currently being used in more than 94% of websites. That means millions of web developers are using JavaScript for web application development. The ease of use and widespread adoption of this programming language is exactly why it continues to become more important to web developers.

THE EVOLUTION OF JAVASCRIPT

- The Ubiquity of Web Browsers – Can you think of a time or place where you aren't within one click of an internet browser? Nope. The web is available everywhere and this makes JavaScript more important and more commonplace. Plus, the widespread adoption ensures dynamic content gets displayed the way it was intended.

THE EVOLUTION OF JAVASCRIPT

- **JavaScript Has More Applications** – Thanks to innovations like Node.js and ReactJS, JavaScript is useful for both front and back end web development. By using cross-platform runtime engines to write server-side code, JavaScript has even more applications and increases efficiencies for web developers.

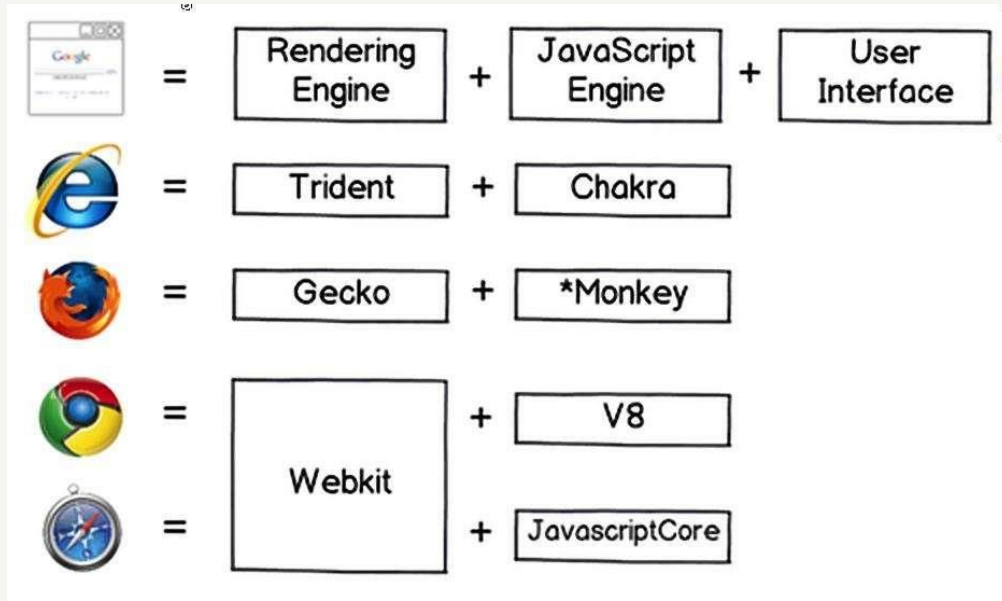
THE EVOLUTION OF JAVASCRIPT

- **The Ubiquity of Web Browsers** – Can you think of a time or place where you aren't within one click of an internet browser? Nope. The web is available everywhere and this makes JavaScript more important and more commonplace. Plus, the widespread adoption ensures dynamic content gets displayed the way it was intended.

THE EVOLUTION OF JAVASCRIPT

- **JavaScript Has More Applications** – Thanks to innovations like Node.js and ReactJS, JavaScript is useful for both front and back end web development. By using cross-platform runtime engines to write server-side code, JavaScript has even more applications and increases efficiencies for web developers.

JS engines



How do engines work?

- Engines are complicated. But the basics are easy.
- The engine (embedded if it's a browser) reads (“parses”) the script.
- Then it converts (“compiles”) the script to the machine language.
- And then the machine code runs, pretty fast.
- The engine applies optimizations at each step of the process. It even watches the compiled script as it runs, analyzes the data that flows through it, and further optimizes the machine code based on that knowledge.

What can in-browser JavaScript do?

- JavaScript's capabilities greatly depend on the environment it's running in. For
- instance, Node.js supports functions that allow JavaScript to read/write arbitrary files, perform network requests, etc.
- Add new HTML to the page, change the existing content, modify styles.

What can in-browser JavaScript do?

- React to user actions, run on mouse clicks, pointer movements, key presses.
- Send requests over the network to remote servers, download and upload files (so- called AJAX and COMET technologies).
- Get and set cookies, ask questions to the visitor, show messages.
- Remember the data on the client-side (“local storage”).

What CAN'T in-browser JavaScript do?

- JavaScript's abilities in the browser are limited for the sake of the user's safety. The aim is to prevent an evil webpage from accessing private information or harming the user's data.
- Examples of such restrictions include:

What CAN'T in-browser JavaScript do?

- JavaScript on a webpage may not read/write arbitrary files on the hard disk, copy them or execute programs. It has no direct access to OS system functions.
- Modern browsers allow it to work with files, but the access is limited and only provided if the user does certain actions, like “dropping” a file into a browser window or selecting it via an `<input>` tag.

What CAN'T in-browser JavaScript do?

- There are ways to interact with camera/microphone and other devices, but they require a user's explicit permission. So a JavaScript-enabled page may not sneakily enable a web-camera, observe the surroundings and send the information to the NSA.

What makes JavaScript unique?

- There are at least three great things about JavaScript:
 - Full integration with HTML/CSS.
 - Simple things are done simply.
 - Support by all major browsers and enabled by default.
- JavaScript is the only browser technology that combines these three things.

What makes JavaScript unique?

- That's what makes JavaScript unique. That's why it's the most widespread tool for creating browser interfaces.
- That said, JavaScript also allows to create servers, mobile applications, etc.

Languages “over” JavaScript

- The syntax of JavaScript does not suit everyone's needs. Different people want different features.
- That's to be expected, because projects and requirements are different for everyone.
- So recently a plethora of new languages appeared, which are transpiled (converted) to JavaScript before they run in the browser.
- Modern tools make the transpilation very fast and transparent, actually allowing developers to code in another language and auto-converting it “under the hood”.

Languages “over” JavaScript

- Examples of such languages:
- CoffeeScript is a “syntactic sugar” for JavaScript. It introduces shorter syntax, allowing us to write clearer and more precise code. Usually, Ruby devs like it.
- TypeScript is concentrated on adding “strict data typing” to simplify the development and support of complex systems. It is developed by Microsoft.

Languages “over” JavaScript

- Flow also adds data typing, but in a different way. Developed by Facebook.
- Dart is a standalone language that has its own engine that runs in non-browser environments (like mobile apps), but also can be transpiled to JavaScript. Developed by Google.
- There are more. Of course, even if we use one of transpiled languages, we should also know JavaScript to really understand what we're doing.

Summary

- JavaScript was initially created as a browser-only language, but is now used in many other environments as well.
- Today, JavaScript has a unique position as the most widely-adopted browser language with full integration with HTML/CSS.
- There are many languages that get “transpiled” to JavaScript and provide certain features. It is recommended to take a look at them, at least briefly, after mastering JavaScript.

JavaScript Uses in Web Designing





TOPSTECHNOLOGIES

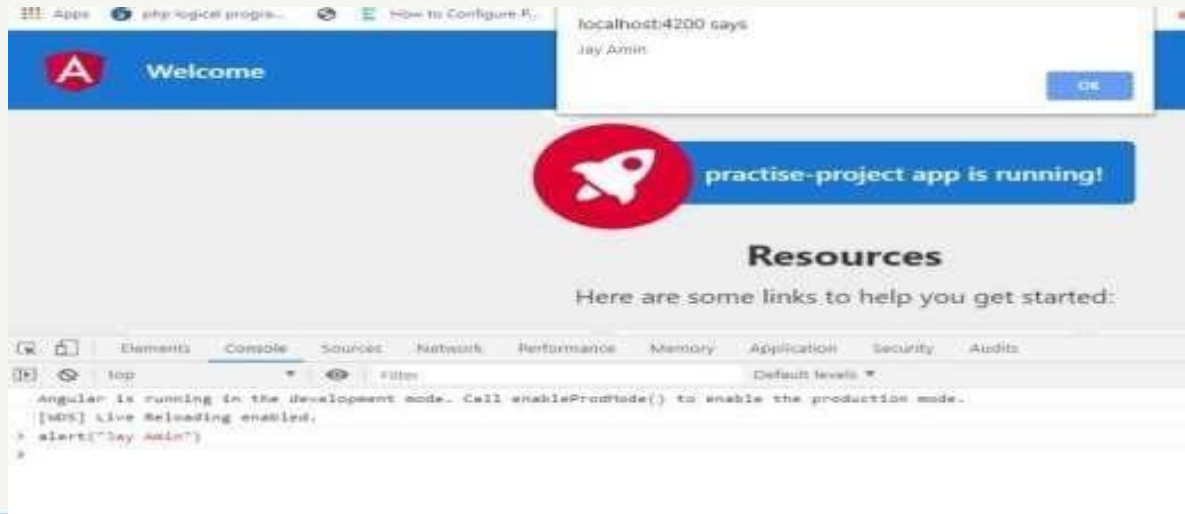
Training | Outsourcing | Placement | Study Abroad

Developer console

F12

or

Right click inspect element



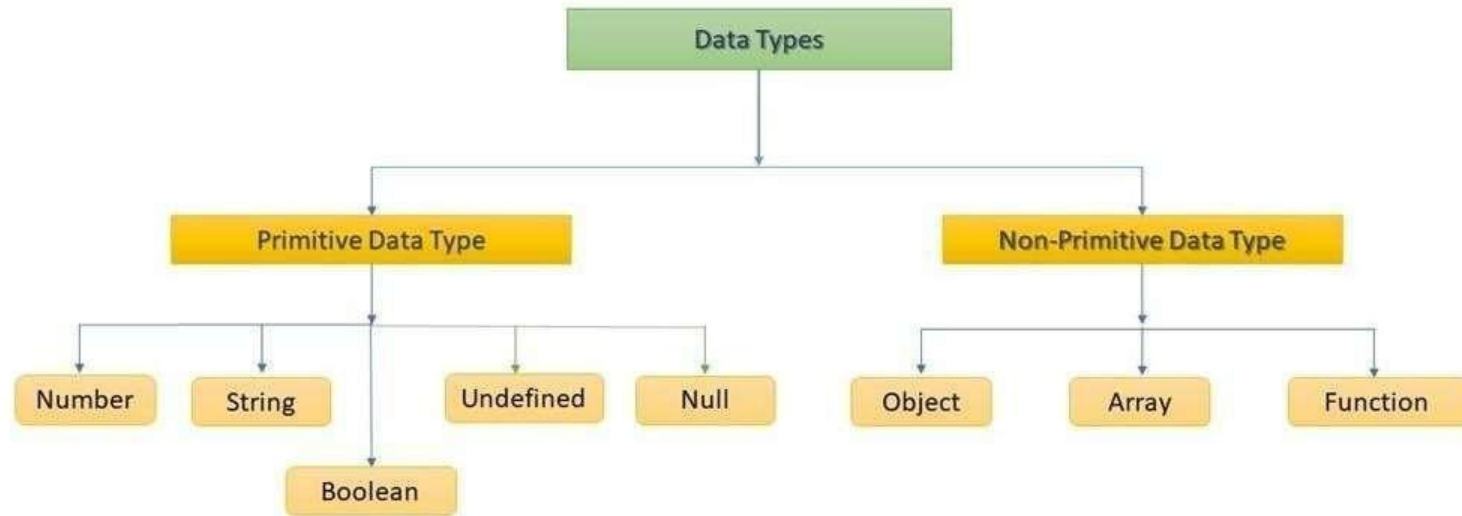
Developer console

- But in the browser, users don't see errors by default. So, if something goes wrong in the script, we won't see what's broken and can't fix it.
- Most developers lean towards Chrome or Firefox for development because those browsers have the best developer tools. Other browsers also provide developer tools, sometimes with special features, but are usually playing "catch-up" to Chrome or Firefox. So most developers have a "favorite" browser and switch to others if a problem is browser-specific.

Developer console

- Developer tools are potent; they have many features. To start, we'll learn how to open them, look at errors, and run JavaScript commands.

Data Types



Javascript place in Html

Placing `<script>` in the `<head>`:

- **How it works:** The script is loaded before the HTML content, meaning that the browser will download and execute the script before rendering the body content.
- **Drawback:** This can block the rendering of the page because the browser must wait for the script to load and execute before displaying any content. This could lead to slower page load times, especially for larger scripts.
- **When to use:** Use this method when the script needs to execute before any content is displayed (e.g., setting up variables, configurations, or making requests that are required for the initial display).

Javascript place in Html

Placing `<script>` in the `<body>` :

- **How it works:** The script is loaded after the HTML content is loaded. This means the browser can display the page content first while the script loads in the background.
- **Advantage:** This method improves page load performance since the browser can render the content while the script is still downloading. Once the HTML is rendered, the script is executed.
- **Best practice:** This is the most common approach because it ensures that the page's content appears quickly, and the script executes afterward without blocking rendering.

Using the async and defer attributes:

- **async attribute:** If placed in the <head>, the script will be fetched in parallel to the HTML parsing, but it will execute as soon as it's downloaded, possibly before the page has finished rendering.
- **defer attribute:** If placed in the <head>, it ensures that the script is executed only after the HTML document has been fully parsed (but before the DOMContentLoaded event is fired).

<script src="script.js" defer></script>

What is a JavaScript Loop?

- **Entry Controlled Loops:** Any loop where we check the test condition before entering the loop is an entry controlled loop. In these loops, the test condition determines whether the program will enter the loop or not. These include for, while, etc.
- **Exit Controlled Loops:** Any loop where we check the test condition after the statements are executed once is an exit controlled loop. In these loops, the test condition determines whether or not the program will exit the loop. This category includes do...while loop.

JavaScript Loops

A **loop** is a programming construct that repeats a block of code multiple times based on a specific condition or set of conditions. In JavaScript, loops allow you to automate repetitive tasks without needing to write the same code repeatedly.

Why Do We Use Loops?

Loops are essential for handling tasks that require repetition. They are used for:

1. **Iterating through collections:** You might want to process each item in an array, object, or other iterable collections.
2. **Repeating a task:** When you need to execute the same action multiple times, like summing numbers, applying a function to each element, or making multiple attempts to meet a certain condition.
3. **Efficiency:** Loops reduce the need for repetitive code, making your program cleaner and more maintainable.

Types of loops

1. For Loop
2. While Loop
3. Do-While Loop
4. For...in Loop
5. For...of Loop

For loop

The **for loop** is used when you know the exact number of times you want to repeat a block of code.

syntax

```
for (initialization; condition; update) {  
    // Code to be executed  
}
```

For loop

Initialization: You define and initialize a variable (often a counter) that controls the loop.

Condition: This is a test expression that is evaluated before each iteration. If the condition is **true**, the loop continues.

Update: The loop variable is updated (usually incremented or decremented) after every iteration.

[Click here for practical](#)

while loop

The **while** loop runs as long as the given condition is **true**. It checks the condition **before** each iteration.

Syntax:

```
while (condition)
{
    // Code to be executed
    // updation
}
```

[Click here for practical](#)

do.. while loop

The **do...while** loop is similar to the **while** loop, but the key difference is that the condition is checked **after** the code is executed. This guarantees that the code will run at least once.

Syntax:

```
do
{
    // Code to be executed
} while (condition);
```

[Click here for practical](#)

For .. in loop

The **for...in loop** is used to loop through the **keys** (or properties) of an **object**. It gives you access to the property names, but not the values directly.

Syntax:

```
for (let key in object) {  
    // Code to be executed  
}
```

For .. in loop

- This loop is specifically used for iterating over **objects** (not arrays).
- On each iteration, the **key** variable holds the name of the current property

[Click here for practical](#)

For .. of loop

- The **for...of loop** is used to loop through **values** in an **iterable** (like an array, string, or other iterable objects).

Syntax

```
for (let value of iterable) {  
    // Code to be executed  
}
```



For .. of loop

Unlike `for...in`, which gives you the keys, `for...of` gives you the **values** of the iterable.

This is commonly used to iterate over arrays or strings.

[Click here for practical](#)

Jumping Statement

jumping statements control the flow of execution by either stopping the current loop or skipping the current iteration and continuing with the next one. The two primary jumping statements are **break** and **continue**.

These statements are used within loops or switch cases to alter the normal flow of execution.

Types of Jumping statements

1. Break Statement
2. Continue Statement
3. Labeled Statement
4. return
5. throw

break Statement

The break statement is used to exit a loop or a switch statement prematurely, meaning it will stop the loop or switch case and transfer control to the next statement after the loop or switch.

Usage of break:

- In loops: The break statement stops the loop entirely, regardless of whether the loop condition is true or not.
- In switch: The break statement exits the switch case once the matched block of code is executed.

[Click here for practical](#)

Continue Statement

The continue statement is used to skip the current iteration of a loop and move to the next iteration. It doesn't stop the loop entirely, but it prevents the rest of the code inside the loop from executing for the current iteration.

Usage of continue:

- In loops: It can be used inside for, while, or do...while loops to skip certain iterations based on a condition.
- It does not work in switch statements: You cannot use continue to skip to the next case in a switch.

[Click here for practical](#)

Labeled statement

- A labeled statement in JavaScript is a way to assign a name (a label) to a specific statement or block of code. This label can then be used with control flow statements like break or continue to control the flow of execution more specifically, especially in complex loops or nested blocks

[Click here for practical](#)

Return statement

The return statement is used to exit a function and optionally return a value to the caller. It stops the execution of the function and can send a value back to the function's caller.

Usage of return:

- In functions: The return statement terminates the function's execution and optionally returns a value.
- In loops: A return inside a function (even if inside a loop) causes the function to exit immediately, so the loop will not continue.

[Click here for practical](#)

Throw statement

The **throw** statement is used to **raise an exception** in JavaScript. It allows you to create custom error messages or objects that can be caught by a **try...catch** block for exception handling. When **throw** is executed, the normal flow of the program is interrupted, and control is transferred to the nearest **catch** block.

- **Exception Handling:** **throw** is used to raise exceptions, which can then be handled using **try...catch** blocks.
- You can throw an error message or custom error object.

[Click here for practical](#)

Function

- A **function** in JavaScript is a block of reusable code designed to perform a specific task. Functions help in organizing code into logical blocks, making it more readable, reusable, and maintainable. JavaScript functions allow you to group a set of instructions together, and you can call these instructions whenever needed.
- In JavaScript, functions can accept inputs, perform some operations, and return a result.

Function

- Function declaration

```
function functionName(parameter1, parameter2, ...) {  
    // Function body: code to execute  
    // You can access parameters and write logic here  
}  
  
functionName(arg1,arg2)
```

[Click here for practical](#)

Function Expression

- A function expression defines a function as part of an expression. The function can be anonymous (without a name) and assigned to a variable.

```
const functionName = function(parameter1, parameter2, ...) {  
    // Function body  
};
```

[Click here for practical](#)

Arrow Function

Arrow Function (ES6)

Arrow functions, introduced in ECMAScript 6 (ES6), provide a shorter syntax for writing functions. The syntax is:

```
const functionName = (parameter1, parameter2) => {  
    // code to be executed  
};
```

[**Click here for practical**](#)

Higher-Order Functions

A higher-order function is a function that takes one or more functions as arguments, returns a function, or both.

These kinds of functions allow for more flexibility and abstraction in your code, and are commonly used in functional programming. Common examples of higher-order functions in JavaScript include `map()`, `filter()`, `reduce()`, and `forEach()`.

1) **Higher-Order Function that Takes a Function as an Argument**

```
function higherOrderFunction(fn) {  
  return fn();  
}
```

```
function sayHello() {  
  return "Hello, World!";  
}
```

```
console.log(higherOrderFunction(sayHello)); // "Hello, World!"
```

[Click here for practical](#)

2) **Higher-Order Function that Returns Another Function**

```
function multiplyBy(factor) {  
  return function(number) {  
    return number * factor;  
  };  
}
```

```
const double = multiplyBy(2);  
console.log(double(5)); // 10
```

```
const triple = multiplyBy(3);  
console.log(triple(5)); // 15
```

[Click here for practical](#)

Function Scope and Closures

Scope refers to the visibility of variables inside different parts of a program.

Closures are functions that "remember" the scope in which they were created, even after the outer function has finished executing

Scope

scope refers to the accessibility or visibility of variables, functions, and objects in some particular part of the code.

JavaScript uses lexical scoping, which means that the scope of a variable is determined by its position in the code at the time it's written.

Types of scope

- 1) Global Scope
- 2) Function Scope
- 3) Block Scope
- 4) Lexical Scope

Global Scope

Variables declared outside of any function or block are said to have global scope, which means they are accessible throughout the entire script.

[*Click here for practicals*](#)

Global variables and Memory management

Global variables persist throughout the lifetime of the page or script. This can lead to memory issues if too many global variables are used, as they are never garbage-collected until the script ends. It's a good idea to minimize the use of global variables when possible.

// If you have many global variables, they will stay in memory throughout the page's lifecycle

```
var global1 = "Global variable 1";  
var global2 = "Global variable 2";
```

```
// and so on...
```

Global Scope in Modules (ES6)

Starting from ES6 (ECMAScript 2015), JavaScript introduced **modules**, which allow you to split your code into smaller, reusable pieces. Code inside modules is **not** in the global scope by default.

Each module has its own scope, which prevents polluting the global namespace.

Global Scope in Modules (ES6)

// file1.js

```
let moduleVariable = "I am local to this module";  
export { moduleVariable };
```

// file2.js

```
import { moduleVariable } from './file1.js';  
console.log(moduleVariable); // Output: I am local to this module
```

Function Scope

Function scope means that variables declared inside a function are only accessible within that function.

These variables are **local** to the function and are not visible or accessible from outside the function.

[Click here for practical](#)

Function Scope

Function Scope with **var** (ES5 and below)

Before ES6, the var keyword was used to declare variables.

var has function scope, which means that variables declared with var are scoped to the function, regardless of where in the function they are declared (even if they are declared inside a block like if or for).

[Click here for example](#)

Function Scope

let and const (ES6 and beyond)

In ES6 (ECMAScript 2015) and later, JavaScript introduced let and const keywords to declare variables. Unlike var, both let and const have **block scope**. This means that variables declared with let or const are limited to the block (such as inside an if statement or loop) in which they are declared.

[Click here for example](#)

Block Scope

block scope refers to the concept that variables declared within a block (i.e., within curly braces `{}`) are only accessible within that block. This behavior is enforced with the `let` and `const` keywords (introduced in ES6), and it applies to any block, such as loops, conditionals, or even standalone blocks.

[Click here for example](#)

Block Scope

let and const in block scope

Before ES6, JavaScript only had function scope with **var**.

Variables declared with **var** inside a block (like an **if** statement or a loop) were still accessible outside of the block, which led to potential issues.

ES6 introduced **block-scoping** with **let** and **const**.

[Click here for let example](#)

[Click here for const example](#)



Block Scope

In loop statements and conditional statements

One of the most common scenarios where block scoping is important is inside loops and conditionals. With **let** and **const**, you can have **block-scoped variables** within loops or **if** statements, preventing unwanted side effects when variables are reused.

[Click here for let example](#)

Why is Block Scope important ?

Block scope helps solve several issues that arise from function-scoped variables (**var**):

- **Avoiding Variable Leaks:** Block scoping prevents variables from being accidentally accessed outside of their intended scope. This makes your code more predictable and safer by reducing the risk of name conflicts and unintended variable sharing.
- **Improved Control in Loops:** Using **let** and **const** in loops ensures that the loop variable is not accessible outside the loop, making it easier to reason about the code and avoid errors caused by reusing loop variables.
- **Safer Conditional Logic:** Block scoping ensures that variables inside an **if** or **else** block are not accidentally used outside of the block, which is helpful when writing complex conditional logic.

lexical scope

Lexical scope (also known as static scope) is the scope (visibility) of variables that is determined at the time the code is written, not when it's executed. This means that JavaScript uses the position of a function in the code to determine which variables are accessible to it.

In JavaScript, functions have access to variables that are declared within them and outside them in the scope chain (lexically), based on where the function is defined. Importantly, the scope is determined by where the function is declared, not by where the function is called.

lexical scope

Inner functions can access variables from their **outer functions** (lexically scoped).

Outer functions cannot access variables from their **inner functions**.

Variables are looked up from the current scope, and if not found, from its outer (parent) scopes

[Click here for example](#)



lexical scope

Outer Functions Cannot Access Inner Variables

innerFunction() can access **outerVar** because it is inside **outerFunction()**.

But **outerFunction()** **cannot** access **innerVar** because **innerVar** is declared inside **innerFunction()**, and variables in inner functions are not accessible to outer functions.

[Click here for practical](#)

closure

What is a Closure?

A **closure** is a function that **remembers** its **lexical environment**, i.e., it "remembers" the variables that were in scope when the function was created, even if the function is executed outside of that scope.

In simpler terms, a closure is a function that has access to its own scope, the outer function's scope, and the global scope, even after the outer function has finished executing.

closure

Key Characteristics of Closures:

1. A closure allows a function to access variables from its outer function even after the outer function has finished executing.
2. A closure can "remember" the environment in which it was created (this includes variables that were in scope at the time of creation).

[Click here for practical](#)

closure

How Closures Work

When you create a function inside another function, the inner function has access to the variables from the outer function.

If you return the inner function, the inner function retains access to the outer function's variables, even after the outer function has executed.

This is because JavaScript keeps track of the variables in the scope where the function was defined (this is the closure).

[Click here for practical](#)



Array

An **array** in JavaScript is a data structure used to store multiple values in a single variable. Arrays are often used to group related data together.

Unlike traditional variables which hold a single value, arrays allow you to store collections of values, such as numbers, strings, or even other arrays or objects.

Key Characteristics of Arrays

Indexed Collection: Arrays in JavaScript are zero-indexed, meaning the first element is at index 0, the second element at index 1, and so on.

Ordered: The elements in an array are ordered, meaning the position of each element matters.

Dynamic Size: JavaScript arrays are dynamically sized, meaning the size of an array can change during runtime (i.e., you can add or remove elements).

Heterogeneous: An array can contain elements of different types, such as numbers, strings, objects, or even other arrays.

Creating an Array

Arrays in JavaScript can be created in two main ways:

1. Using Array Literals (Recommended)

```
let fruits = ["Apple", "Banana", "Cherry"];
```

This is the most common and preferred way to create an array. You enclose the elements in square brackets (`[]`), separated by commas.

[Click here for practical](#)

Creating an Array

2. Using the **Array** Constructor

```
let fruits = new Array("Apple", "Banana", "Cherry");
```

This creates an array, but the syntax is less concise and less common than using array literals.

[Click here for practical](#)

Array Methods

`push()` - Add an element to the end

[*Click here for practical*](#)

`pop()` - Remove the last element

[*Click here for practical*](#)

`shift()` - Remove the first element

[*Click here for practical*](#)

`unshift()` - Add an element to the beginning

[*Click here for practical*](#)

Array Methods

`length` - Get the number of elements

[*Click here for practical*](#)

`indexOf()` - Find the index of an element

[*Click here for practical*](#)

`splice()` - Add or remove elements at specific positions

[*Click here for practical*](#)

`slice()` - Create a shallow copy of a portion of the array

[*Click here for practical*](#)

Array Methods

`forEach()` - Loop through each element

[*Click here for practical*](#)

`map()` - Create a new array by applying a function to each element

[*Click here for practical*](#)

`filter()` - Filter elements based on a condition

[*Click here for practical*](#)

`reduce()` - Reduce the array to a single value

[*Click here for practical*](#)

Multidimensional Arrays

JavaScript arrays can also be multidimensional, meaning an array within an array:

[*Click here for practical*](#)

Array Destructuring

JavaScript allows you to use destructuring to unpack values from arrays:

[*Click here for practical*](#)

Array.from()

Array.from() is a built-in method in JavaScript that creates a new, shallow-copied array instance from an array-like or iterable object

Converting a string to an array: [Click here for practical](#)

Converting set into array : [Click here for practical](#)

Using of mapFunction : [Click here for practical](#)

Array-like Objects: These include objects that have a length property and indexed elements, like arguments, NodeList (from DOM queries), or strings.

[Click here for practical](#)

Object

An object is a collection of related data and/or functionality. It is one of the most fundamental data structures and allows you to group together various values under a single entity. These values can be of any data type, including strings, numbers, arrays, and even other objects.

properties

Properties (Key-Value Pairs): Objects consist of properties, which are key-value pairs. The key is a string (also called the property name), and the value can be any data type (string, number, array, function, etc.).

```
const person = {  
  name: "pari",    // key is 'name' and value is pari  
  age: 20,         // key is 'age' and value is 30  
  isEmployed: true // key is 'isEmployed' and value is true  
};
```

[Click here for practical](#)

methods

In addition to data, objects can also have methods. Methods are functions that are associated with an object and typically operate on the object's properties.

```
const person = {  
  name: "shreya",  
  age: 30,  
  greet: function() {  
    console.log("Hello, " + this.name);  
  }  
};  
person.greet(); // Output: Hello, shreya
```

[Click here for practical](#)

Creating Object

There are 3 ways to create object

1) Using Object Literals (Recommended for most cases):

```
const mobile = {  
  brand: "Samsung",  
  model: "s24",  
  year: 2024  
};
```

```
console.log(mobile);
```

[Click here for practical](#)

2) Using the new Object() Syntax:

```
const mobile = new Object();  
mobile.brand = "Samsung";  
mobile.model = "S24";  
mobile.year = 2024;  
  
console.log(mobile);
```

[Click here for practical](#)



3) Using a Constructor Function (typically for creating multiple instances of similar objects):

```
function Mobile(brand, model, year) {  
  this.brand = brand;  
  this.model = model;  
  this.year = year;  
}
```

```
const myMobile = new Mobile("Samsung", "S24", 2024);  
console.log(myMobile);
```

[Click here for practical](#)

JavaScript Objects

In JavaScript, almost "everything" is an object.

- Booleans can be objects (if defined with the new keyword)
- Numbers can be objects (if defined with the new keyword)
- Strings can be objects (if defined with the new keyword)
- Dates are always objects
- Maths are always objects
- Regular expressions are always objects
- Arrays are always objects
- Functions are always objects
- Objects are always objects

All JavaScript values, except primitives, are objects.

Accessing Objects

Dot Notation

Dot notation is the most common and straightforward way to access an object's properties

e.g. `console.log(person.name);`

Bracket Notation

Bracket notation is often used when the property name is dynamic (e.g., stored in a variable) or when the property name contains spaces or special characters.

e.g. `console.log(person["name"]);`

[Click here for practical](#)



Add - Delete Properties

Adding Properties

```
person.country = "USA";
```

```
console.log(person.country); // "USA"
```

Delete Properties

```
delete person.age;
```

```
console.log(person.age); // undefined
```

Object Destructuring

Destructuring is a convenient way to extract values from an object into variables.

[*Click here for practical*](#)

Object Methods

- `Object.keys(obj)` – Returns an array of the object's property names (keys).
- `Object.values(obj)` – Returns an array of the object's property values.
- `Object.entries(obj)` – Returns an array of arrays, each containing a key-value pair.

[Click here for practical](#)

Object.freeze() and Object.seal()

JavaScript provides methods to make objects immutable or prevent new properties from being added.

- **Object.freeze(obj)** – Freezes an object, preventing new properties from being added and existing properties from being modified or deleted.
- **Object.seal(obj)** – Seals an object, preventing new properties from being added but allows existing properties to be modified.

[Click here for practical](#)

Prototype Chain

Every JavaScript object has an internal property called `[[Prototype]]`, which is used for inheritance. You can access the prototype of an object using `Object.getPrototypeOf()`.

[Click here for practical](#)

String in JS

A string is a sequence of characters used to represent text. Strings are one of the most commonly used data types in JavaScript and are integral to web development tasks such as manipulating text, processing user input, and working with data formats like JSON, HTML, etc.

Create String

There are two ways to define string

1) ***Using String Literal (Single or Double Quotes) - practical***

string by enclosing a sequence of characters in single (') or double (") quotes.

2) ***Using Template Literals (Backticks) - practical***

Template literals, introduced in ECMAScript 6 (ES6), allow you to create strings with more flexibility, such as including expressions inside the string. Template literals are enclosed by backticks (`).

String Properties and Methods

Accessing characters : can access individual characters in a string using bracket notation.

length: Returns the number of characters in the string.

toUpperCase() : Converts a string to uppercase:

toLowerCase() Converts a string to lowercase:

substring(start, end) Extracts a part of a string:

replace(oldValue, newValue) - Replaces part of a string:

split(separator) - Splits a string into an array of substrings:

trim() - Removes whitespace from both ends of a string:

[Click here for practical](#)



TOPSTECHNOLOGIES

Training | Outsourcing | Placement | Study Abroad

DOM

Document Object Model

DOM

JavaScript is a programming interface for web documents. It represents the structure of a web page as a tree of objects, where each object corresponds to a part of the page (like elements, attributes, text, etc.).

This allows JavaScript to interact with and manipulate the content, structure, and style of web pages dynamically.

Document: The **document** object represents the entire HTML document and serves as the entry point for accessing and manipulating elements within the page.

Node: The DOM represents every part of an HTML document as a **node** (an element, attribute, or piece of text). The most common types of nodes are:

- **Element nodes** (e.g., `<div>`, `<p>`)
- **Text nodes** (e.g., the content inside `<p>`)
- **Attribute nodes** (e.g., `class="myClass"` in an HTML element)

DOM Tree Structure: The structure of the DOM is hierarchical, and you can think of it as a tree where:

- The **document** is the root.
- Each element is a node branching off from the root or from other nodes.
- Text and attributes are also nodes that are attached to their respective elements.

DOM Manipulation: JavaScript can interact with the DOM to modify the page dynamically, such as adding or removing elements, changing styles, handling events, and much more.

[Click Here for practicals](#)

getElementById()

- Purpose: Selects an element by its unique id attribute.
- Description: This is the most efficient method when selecting a single element because id attributes are required to be unique within a page.

Syntax :

```
var element = document.getElementById("id");
```

- Note: getElementById() returns a single element, even though multiple elements with the same id would be considered invalid according to HTML standards.

getElementsByClassName()

- Purpose: Selects all elements with a specified class name.
- Description: This method returns a live HTMLCollection (a live list that automatically updates when the document changes) of all elements with the given class name. Multiple elements can have the same class name.

Syntax :

```
var elements = document.getElementsByClassName("className");
```

[Click here for example](#)

getElementsByTagName()

- Purpose: Selects all elements with a specified tag name (e.g., <div>, <p>, <a>).
- Description: Similar to getElementByClassName(), this method returns a live HTMLCollection of elements with the specified tag name.

Syntax:

```
var elements = document.getElementsByTagName("tagName")
```


Promises

JavaScript Promises make handling asynchronous operations like API calls, file loading, or time delays easier. Think of a Promise as a placeholder for a value that will be available in the future. It can be in one of three states

Pending: The task is in the initial state.

Fulfilled: The task was completed successfully, and the result is available.

Rejected: The task failed, and an error is provided.

[Click here for practical](#)

Promises

`resolve(value)`: Marks the promise as fulfilled and provides a result.

`reject(error)`: Marks the promise as rejected with an error.

[Click here for practical](#)

Promises

JavaScript Promises make handling asynchronous operations like API calls, file loading, or time delays easier. Think of a Promise as a placeholder for a value that will be available in the future. It can be in one of three states

Pending: The task is in the initial state.

Fulfilled: The task was completed successfully, and the result is available.

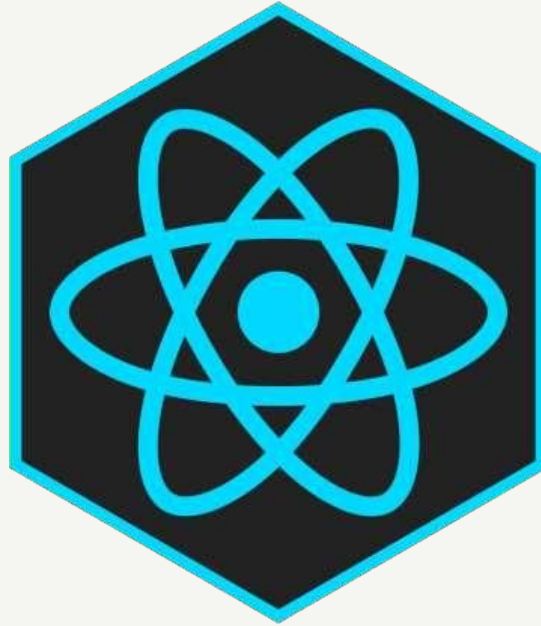
Rejected: The task failed, and an error is provided.

[Click here for practical](#)



TOPSTECHNOLOGIES

Training | Outsourcing | Placement | Study Abroad



ReactJS

Module-9

React - Components, State, Props

Introduction

React.js (often just called React) is an open-source JavaScript library used for building user interfaces, particularly for single-page applications where you need fast, dynamic updates without reloading the whole page.

It was developed by Facebook and is widely used for creating interactive and reusable UI components.

What is ReactJS

ReactJS is an open source JavaScript library used to develop User Interfaces.



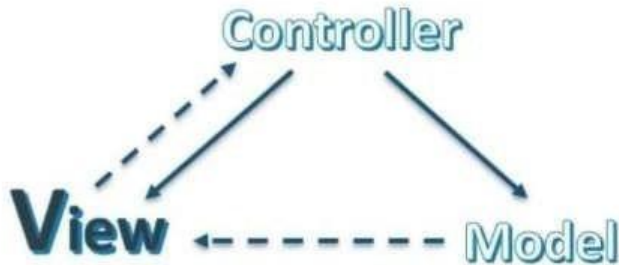
facebook.



ReactJS was introduced by Facebook on May, 2013. It was open sourced in March, 2015.

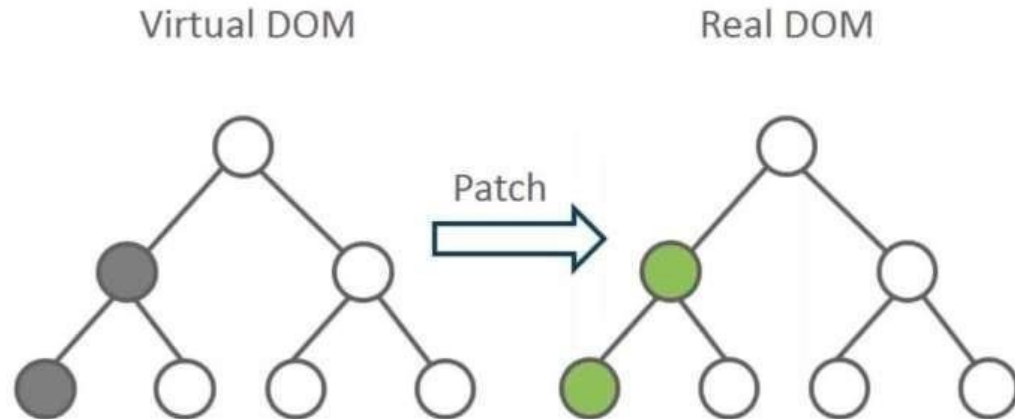
What is ReactJS

ReactJS is concerned with the components that utilizes the expressiveness of JavaScript with a HTML – like template syntax.

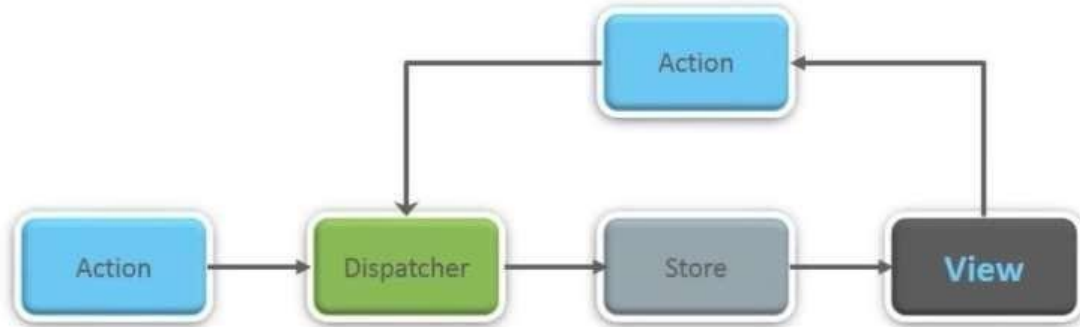


It is basically the View in MVC
(Model-**View**-Controller)

3D Aspects Of React Virtual Dom

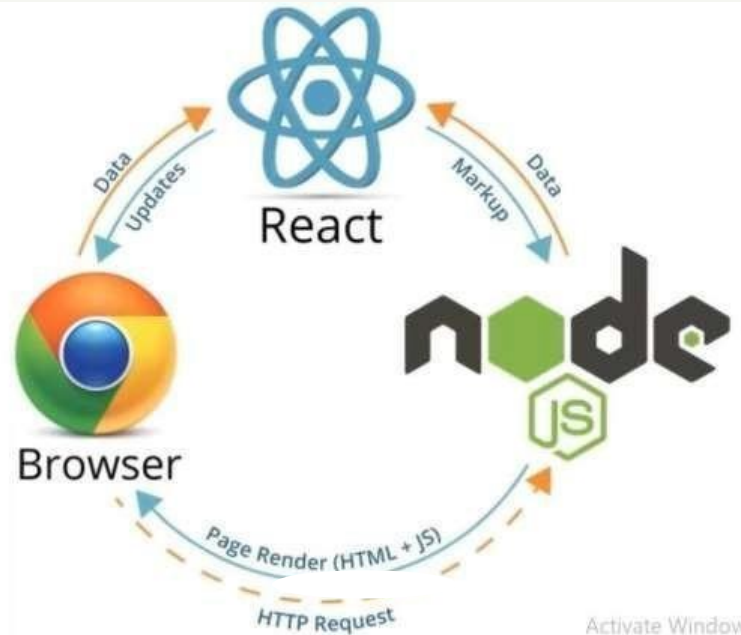


3D Aspects Of React Unidirectional Data Binding



One Way Data Binding

3D Aspects Of React Server-side Rendering



features of React

Component-Based Architecture: React allows developers to build UIs as a collection of components, each representing a part of the user interface. Components can be reused and nested within one another, making development more modular and maintainable.

Virtual DOM: React uses a Virtual DOM (Document Object Model), which is a lightweight copy of the real DOM. When changes are made to the UI, React updates the Virtual DOM first, then efficiently compares it with the real DOM and only applies the necessary changes, improving performance.

Declarative Syntax: React uses a declarative approach to define UI components. Instead of telling the app how to update the UI step by step, you describe how the UI should look based on the current state. React takes care of updating the DOM.

features of React

JSX (JavaScript XML): React uses JSX, a syntax extension that allows you to write HTML-like code within JavaScript. JSX makes it easy to visualize the structure of your UI while using JavaScript logic.

Unidirectional Data Flow: React follows a one-way data flow where the data flows from parent components to child components. This makes it easier to understand and debug how data changes within an application.

State and Props: Components can have **state** (internal data that can change over time) and **props** (external data passed to components). React automatically updates the UI when the state or props change.

DOM

In React, the **DOM** (Document Object Model) is a crucial concept because React interacts with the DOM to update the UI.

The DOM is a programming interface for web documents. It represents the page so that programs can manipulate the structure, style, and content. In a web browser, the DOM is a tree-like structure where each node represents part of the web page (e.g., elements, attributes, text).

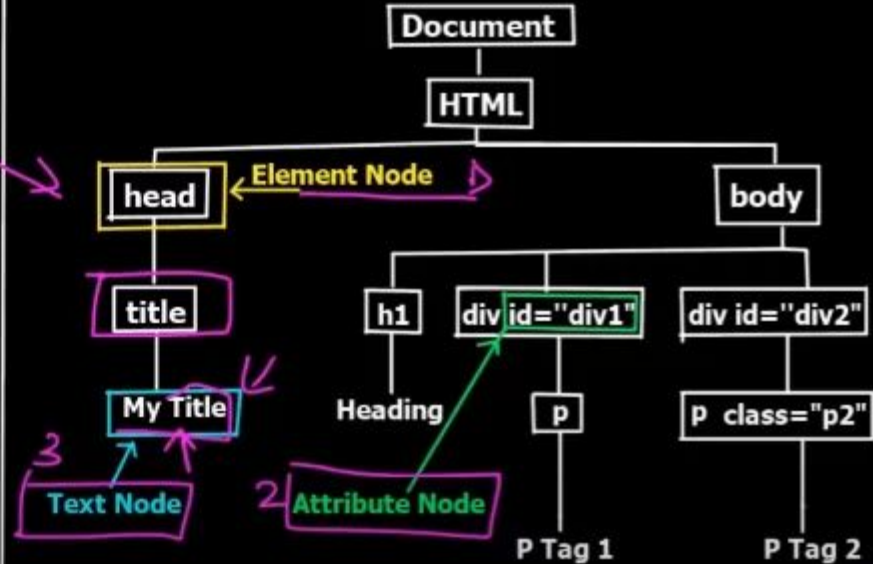


Document Object Model - DOM

HTML Document

```
<html>
  <head>
    <title>My Title</title>
  </head>
  <body>
    <h1>Heading</h1>
    <div id="div1">
      <p>P Tag 1</p>
    </div>
    <div id="div2">
      <p class="p2">P Tag 2</p>
    </div>
  </body>
</html>
```

DOM - JS view



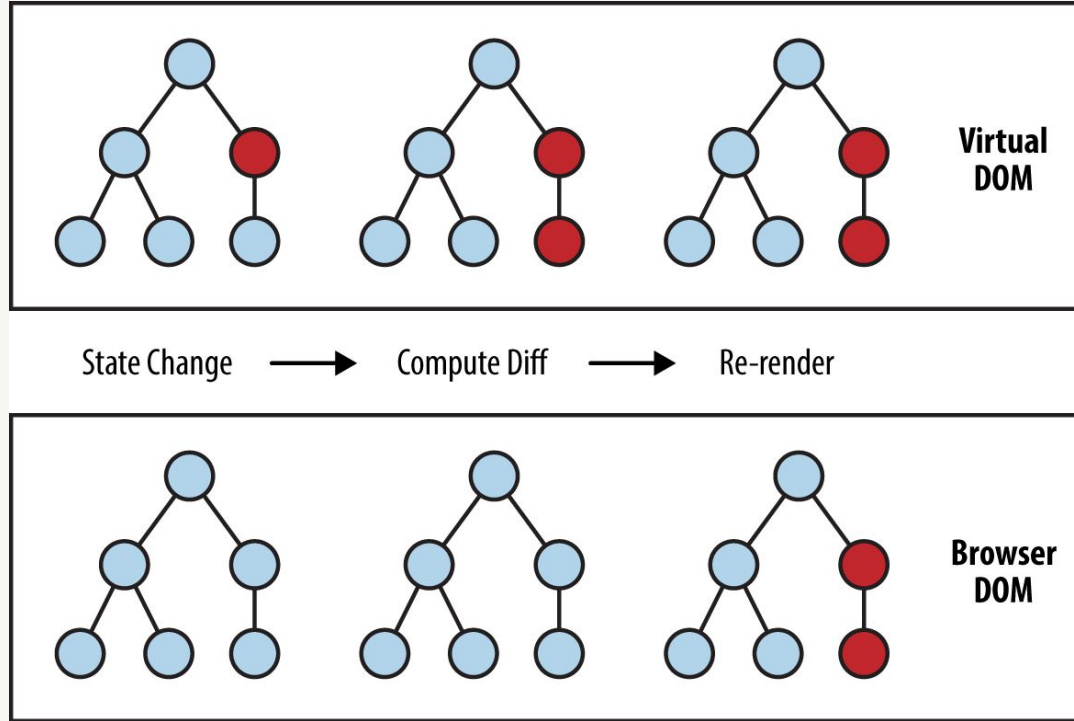
DOM in React

In traditional web development, when you update a web page (like changing an element's text or style), the browser updates the actual DOM. However, direct manipulation of the DOM can be slow, especially for complex applications with many updates, which leads to performance issues.

To solve this, React introduced the concept of the **Virtual DOM**

Virtual DOM

- React keeps a virtual representation of the real DOM in memory. This is a lightweight copy of the actual DOM.
- When the state of a component changes, React first updates the virtual DOM instead of the real DOM directly.
- Then, React compares the virtual DOM with a snapshot of the previous state (a process called **reconciliation**).
- It then calculates the minimal set of changes required to update the real DOM, which makes React's rendering much faster.



SPA :

A single-page application is an app that doesn't need to reload the page during its use and works within a browser

When we navigate between multiple page in react it not change page but it change only dom

Babel : Babel is a JavaScript compiler that can translate markup or programming languages into JavaScript.

In vs code install extension babel and code runner

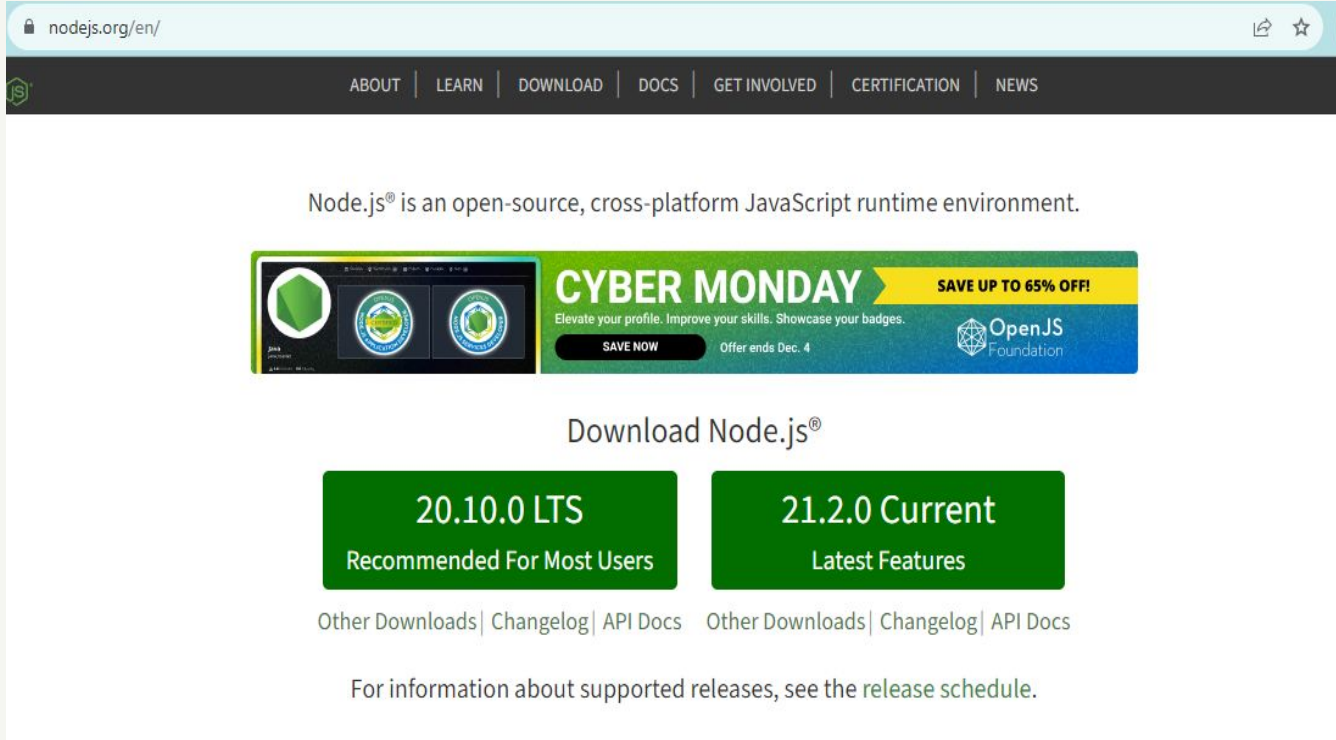


TOPSTECHNOLOGIES

Training | Outsourcing | Placement | Study Abroad

Installation

Install Node :



The screenshot shows the Node.js website. At the top is a navigation bar with links: ABOUT, LEARN, DOWNLOAD, DOCS, GET INVOLVED, CERTIFICATION, and NEWS. Below the navigation bar, a text block states: "Node.js® is an open-source, cross-platform JavaScript runtime environment." Below this is a promotional banner for "CYBER MONDAY" with a yellow bar saying "SAVE UP TO 65% OFF!". The banner includes the text "Elevate your profile. Improve your skills. Showcase your badges." and a "SAVE NOW" button. To the right of the banner is the OpenJS Foundation logo. Below the banner, the text "Download Node.js®" is centered. Underneath are two green buttons: "20.10.0 LTS Recommended For Most Users" and "21.2.0 Current Latest Features". At the bottom, there are links for "Other Downloads | Changelog | API Docs" repeated twice. A final text block at the bottom says: "For information about supported releases, see the [release schedule](#)."

nodejs.org/en/

ABOUT | LEARN | DOWNLOAD | DOCS | GET INVOLVED | CERTIFICATION | NEWS

Node.js® is an open-source, cross-platform JavaScript runtime environment.

CYBER MONDAY **SAVE UP TO 65% OFF!**
Elevate your profile. Improve your skills. Showcase your badges.
SAVE NOW Offer ends Dec. 4 

Download Node.js®

20.10.0 LTS
Recommended For Most Users

21.2.0 Current
Latest Features

Other Downloads | Changelog | API Docs Other Downloads | Changelog | API Docs

For information about supported releases, see the [release schedule](#).

Create React App (CRA) has long been the go-to tool for most developers to scaffold React projects and set up a dev server. It offers a modern build setup with no configuration.

But it increased development and build time when the project size increases

Vite is a next-generation, front-end tool that focuses on speed and performance.

It consists of two major parts:

- 1) A development server
- 2) A Build Command

Check version of node

1) (Traditional installation)

```
#node -v  
#npm install create-react-app  
#npx create-react-app my-app  
#cd my-app  
#npm start
```

npm : node package manager npm is a tool which is used to install packages.

npx : npx is a tool to execute packages.

SWC : SWC (Speedy Web Compiler) is an extensible Rust-based platform for the next generation of fast developer tools.

SWC is used to transform your JavaScript/TypeScript code into a format that the browser can understand. It replaces Babel and TypeScript in the build process

```
PS C:\Users\tops\Documents\GitHub\MERN_STACK\REACT> npm create vite@latest
Need to install the following packages:
  create-vite@5.0.0
Ok to proceed? (y) y
✓ Project name: ... myproject
? Select a framework: » - Use arrow-keys. Return to submit.
> Vanilla
  Vue
  React
  Preact
  Lit
  Svelte
  Solid
  Qwik
  Others
```


2) create new project using vite :

```
#npm init vite@latest my-react-app --template react
```

Ok to proceed ? (y) : y

```
#cd maincontentapp
```

```
#npm install (Install the dependencies required for the project)
```

```
#npm run dev
```



TOPSTECHNOLOGIES

Training | Outsourcing | Placement | Study Abroad



Vite + React

count is 0

Edit `src/App.jsx` and save to test HMR

Click on the Vite and React logos to learn more

React Strict Mode

development tool in React that helps developers write better and more reliable code by highlighting potential problems in the app.

It doesn't affect the production build or performance of the app, but it provides additional checks and warnings during development.

it check warning throughout application during development.
it prevent deprecated Apis

Node Module

node_modules folder is a directory that contains all the dependencies (packages) required for the application to function.

These dependencies are installed through npm (Node Package Manager) or yarn, and the node_modules folder is created when you run npm install or yarn install after setting up a new React project or adding a package to the project

The node_modules folder stores all the third-party libraries, frameworks, and tools that your React application depends on. For example, React itself, react-dom, webpack, babel, and any other libraries you install via npm or yarn will be in this folder.



Project Structure Overview

```
my-react-app/  
├── public/           # Static files (favicon, index.html, etc.)  
├── src/              # Main source code  
│   ├── assets/       # Images, fonts, styles, etc.  
│   ├── components/    # Reusable UI components  
│   ├── pages/         # Page-level components (route-specific)  
│   ├── layouts/       # Layout components (wrappers for pages)  
│   ├── hooks/         # Custom hooks  
│   ├── context/       # React Context API providers  
│   ├── utils/         # Utility functions/helpers  
│   ├── services/      # API calls and external services  
│   ├── store/         # State management (Redux, Zustand, etc.)  
│   ├── routes/        # App routing configuration  
│   ├── App.jsx        # Root component  
│   └── main.jsx       # Entry point (React 18+ uses createRoot)  
└── .env              # Environment variables
```

JSX

JSX (JavaScript XML) is a syntax extension for JavaScript used in React to describe UI components.

It allows you to write HTML-like code within JavaScript, making it easier to visualize the UI structure.

Why Use JSX ?

- It makes UI code **more readable and expressive**.
- It allows **embedding JavaScript expressions** within curly braces `{}`.
- JSX **compiles down to JavaScript** using Babel before rendering in the browser.

[Click here for practicals](#)

JSX Rule ?

- Must have a single parent element
- Use className instead of class
- Self-closing tags for elements without children
- JavaScript expressions inside {}
- Use camelCase for HTML attributes

JSX Compilation

JSX is not valid JavaScript, so Babel compiles it into regular JavaScript:

jsx

```
const element = <h1>Hello, World!</h1>;
```

Compiles to Js

```
const element = React.createElement("h1", null, "Hello,  
World!");
```

Components

A **component** is a **reusable** and **independent** piece of UI in React.

It is like a JavaScript function that returns JSX (UI elements).
Components help build modular and maintainable UIs.

Types of Components

There are two main types of components in React:

1. **Functional Components** (Modern & Recommended)
2. **Class Components** (Older Approach)

Function Component

1. Functional Components (Recommended)

A **functional component** is a JavaScript function that returns JSX. It is simple and does not require a **class**.

[Click here for practical](#)

Advantages of Functional Components:

- **Easier to read & write**
- **Better performance** (no extra overhead like class components)
- Supports **React Hooks** (for state and lifecycle methods)

Class Component

2. Class Components (Older Approach)

Before React Hooks, components were created using ES6 classes. These components extend `React.Component` and have a `render()` method.

[Click here for practical](#)

Disadvantages of Class Components:

- More **verbose** (requires **this** keyword)
- Harder to manage **state** and **lifecycle methods**
- Not recommended for new projects

- **React Class Component Lifecycle**

In React **class components**, lifecycle methods allow you to control **what happens at different stages** of a component's life (from creation to removal).

These lifecycle methods are categorized into **three phases**:



Phase	Method	Purpose
Mounting	<code>constructor()</code>	Initialize state, bind functions
	<code>getDerivedStateFromProps()</code>	Sync state with props (rarely used)
	<code>render()</code>	Render UI
	<code>componentDidMount()</code>	Perform API calls, subscriptions
Updating	<code>getDerivedStateFromProps()</code>	Sync state with new props
	<code>shouldComponentUpdate()</code>	Optimize rendering (return <code>false</code> to prevent re-render)
	<code>render()</code>	Re-render UI
	<code>getSnapshotBeforeUpdate()</code>	Capture values before update (e.g., scroll position)
	<code>componentDidUpdate()</code>	Perform side effects after update
Unmounting	<code>componentWillUnmount()</code>	Cleanup (remove event listeners, clear timers)

Import & Export Components in React

components are usually kept in **separate files** for better organization. To use a component in another file, you need to **export** it and then **import** it.

Exporting Components

There are **two ways** to export a component:

1. **Default Export**
2. **Named Export**

Default Export (Common Approach)

- Used when exporting a **single component** from a file.
- You can **import it with any name** in another file.

Export Syntax :

```
const Greeting = () => {  
    return <h1>Hello, Welcome to React!</h1>;  
};  
export default Greeting; // Default Export
```

Import Syntax :

```
import Greeting from "./Greeting"; // Importing the default export
```

Named Export (For Multiple Components)

- Used when **exporting multiple components** from a file.
- You **must import using the exact name** in `{}`.

Export Syntax :

```
export const Welcome = () => <h1>Welcome to React!</h1>;
```

```
export const Msg = () => <h1>Hello everyone</h1>;
```

Import syntax :

```
import { Welcome, Msg } from "./Messages";
```

Props in React

Props (short for "**Properties**") are a way to pass **data** from one component to another in React.

They allow components to be **dynamic and reusable** by receiving values from their parent components.

Props are immutable (read-only) – a component cannot modify its own props.

Props flow from parent to child – data moves one way (top to bottom).

Props can be of any data type – string, number, boolean, array, object, function, etc.

1) **Passing Props to a Functional Component**

[Click here for practical](#)

2) **Passing Props to a Class Component**

[Click here for practical](#)

3) **Destructuring props**

[Click here for practical](#)

Hook Introduction

Hooks were introduced in **React 16.8** to allow functional components to use **state** and other React features **without writing class components**.

Why Hooks?

Before hooks, React components were either:

- Functional components (stateless, no lifecycle methods).

- Class components (stateful, with lifecycle methods).

Hooks enable stateful logic inside functional components, making code more readable and reusable.

State in React

state is an object that holds data or information about the component. It determines how the component behaves and renders. When the state changes, the component re-renders to reflect the new data.

State in React

state is a built-in feature that allows components to keep track of changing data. It represents the internal state of a component, and when the state changes, React re-renders the component to reflect the updated state.

State is mutable and can be modified using the `setState()` method provided by React.

useState Hook

What is useState?

The useState hook is a built-in React hook that allows functional components to manage state. It lets components store, update, and re-render based on dynamic data without using class components.

Syntax of useState

```
const [state, setState] = useState(initialValue);
```

- **state** → The current state value.
- **setState** → A function to update the state.
- **initialValue** → The default value for the state.

[Click here for practical](#)

State in Function Component

In modern React applications, functional components manage state using the useState hook.

[*Click here for practical*](#)

Update state

1) Direct Update (Simple State)

```
setCount(count + 1);
```

2) Functional Update (When New State Depends on Previous State)

State updates are asynchronous, so for updates based on the previous state, use a function:

```
setCount(prevCount => prevCount + 1);
```

Style

In React, we can style components using multiple approaches, depending on our project needs

- 1) Inline Styles in React
- 2) CSS Stylesheets (.css files)
- 3) CSS Modules (Scoped CSS)
- 4) Styled Components (CSS-in-JS)
- 5) Tailwind CSS (Utility-First CSS Framework)
- 6) Dynamic Styling in React

Style Uses :

Styling Method	Use Case
Inline Styles	Quick, dynamic styles, but not recommended for complex applications.
CSS Files	Traditional approach, good for global styles.
CSS Modules	Best for component-scoped styles in large projects.
Styled Components	Good for scoped styles, dynamic theming, and modern apps.
Tailwind CSS	Fast development with utility classes, great for responsive design.

1) **Inline Styles in React**

React allows us to use inline styles by passing a JavaScript object to the style attribute.

[Click here for practical](#)

- Uses a JavaScript object for styles.
- Properties use camelCase (backgroundColor instead of background-color).
- Useful for dynamic styling but not recommended for large projects due to performance concerns

2) CSS Stylesheets (.css files)

can write styles in an external CSS file and import it into your component.

[Click here for practical](#)

- Keeps styles separate from logic (better maintainability).
- Familiar approach for developers used to traditional CSS.
- Suitable for larger projects with reusable styles.

3) CSS Modules (Scoped CSS)

CSS Modules scope styles to a specific component, preventing conflicts.

[Click here for practical](#)

- Prevents global CSS conflicts by scoping styles to components.
- Class names are unique (e.g., .button_abc123).
- Recommended for large applications where styles need isolation.

4) Styled Components (CSS-in-JS)

Latest react version (2024 is "react": "^19.0.0",)

Styled Components is a popular library that allows you to write CSS directly inside JavaScript.

#npm install styled-components

[Click here for practical](#)

- Styles are written inside JavaScript files.
- Uses template literals (```) for styles.
- Supports dynamic styling with props:

Note : styled-components is not yet fully compatible with React 19, causing the useContext error

Downgrade to the Stable React 18 Version

Ensure your **package.json** looks like this:

```
"dependencies": {  
  "react": "^18.2.0",  
  "react-dom": "^18.2.0",  
  "styled-components": "^6.1.15"  
}
```

Delete **node_modules** and reinstall dependencies.

```
npm cache clean --force  
npm install
```

Restart your development server.

5) Tailwind CSS (Utility-First CSS Framework)

Tailwind CSS is a utility-first framework that provides pre-built classes for rapid styling

[Click here for practical](#)

```
#npm install -D tailwindcss postcss autoprefixer
```

```
#npx tailwindcss init -p
```

6) Dynamic Styling in React

[Click here for practical](#)

- Uses state (useState) to change styles dynamically.
- Inline styles can be modified based on conditions.

Module – 10

React Lists , Hooks , Localstorage , Api Project

Arrays

An array in JavaScript is an ordered list of values. In React, arrays are often used to store lists of items or to manage state when you need to keep track of multiple similar pieces of data (like a list of users or a collection of objects). Arrays in React are also commonly used for rendering dynamic lists of components using `.map()`

[Click here for practical](#)

State with Arrays:

When using arrays in React for state management, the state will typically be set up using `useState`. We can add, remove, or update items in an array.

Immutable Updates: When you want to update an array in React (like adding or removing elements), you should never directly mutate the array (e.g., using `push()`, `splice()`). Instead, create a new array to maintain immutability. React relies on immutability to detect changes and re-render components effectively.

[Click here for practical](#)

Objects

An object in JavaScript is a collection of key-value pairs. In React, objects are commonly used to represent more complex pieces of data, such as the state or props of a component, or for managing individual pieces of data within a component

[Click here for practical](#)

State with Objects

When managing an object in the state, React useState hook allows you to store the object. If you need to update a specific property of the object, you should ensure that you maintain immutability.

[Click here for practical](#)

Arrays are best for managing ordered collections of similar items and are useful for rendering lists.

Objects are useful for managing more complex data, especially when multiple properties belong together (like a user's information).

React requires immutability, so when modifying arrays or objects in state, you should always create a new copy (e.g., using the spread operator or methods like `.concat()` or `.map()`).

You can combine both arrays and objects to manage more complex data structures, like an array of objects, for more dynamic and complex rendering

[Click here for practical](#)

Keys

React uses the key prop to optimize the re-rendering process by efficiently tracking elements in a list. When a list is updated (such as adding or removing items), React needs to figure out how to apply the changes without unnecessarily re-rendering every single item in the list. This is where keys come in.

Without keys, React will rely on the index of the items in the array (in some cases) to track changes. However, this can cause issues, especially if the list is reordered or if items are added or removed, because indexes might no longer uniquely identify the items.

[Click here for practical](#)

Why Not Use the Index as a Key

Using the index as the key can lead to issues in certain situations. For instance:

- **Reordering:** If the list is reordered (e.g., moving items around), React might mistakenly think the items haven't changed, leading to incorrect or unexpected rendering.
- **Adding or Removing Items:** If items are added or removed, React may misidentify which items have changed.

- Always use a **unique and stable** key (preferably an **id**).
- Avoid using **array index** as a key unless the list is static.
- Keys **help React efficiently update the UI** and prevent unnecessary re-renders.

React Hooks

React **Hooks** are special functions that let you **use state and other React features** in functional components.

They were introduced in **React 16.8** to replace class components for managing state and side effects.

1) **useState – Manage State**

The useState hook allows you to add state to a functional component.

[Click here for practical](#)

2) **useEffect** – Handle Side Effects

The `useEffect` hook is used for side effects (e.g., fetching data, updating the DOM, setting timers).

What is a Side Effect?

Side effects are operations that affect something outside the scope of the function being executed. Some examples of side effects in React are:

- Fetching data from an API
- Subscribing to or cleaning up event listeners
- Interacting with the DOM (e.g., updating the title of the page)
- Setting up timers (e.g., setInterval or setTimeout)

Syntax of **useEffect**

```
import { useEffect } from "react";
```

```
useEffect(() => {
```

```
  // Side effect logic here
```

```
});
```

Different Use Cases of useEffect

1. useEffect Without Dependencies (Runs on Every Render)

If you do not provide a dependency array, `useEffect` runs **after every render**.

[Click here for practical](#)

Use case: Debugging or logging values on every render.

Different Use Cases of useEffect

2. useEffect With an Empty Dependency Array [] (Runs Only Once - Like componentDidMount)

When you pass an empty dependency array ([]), useEffect will only run once, after the initial render.

[Click here for practical](#)

Different Use Cases of useEffect

3. useEffect With Dependencies (Runs Only When Dependencies Change)

When dependencies are provided, useEffect runs only when those values change.

[Click here for practical](#)

`useEffect(() => { ... })` → Runs on every render.

`useEffect(() => { ... }, [])` → Runs once when mounted (like `componentDidMount`).

`useEffect(() => { ... }, [dependency])` → Runs when the dependency changes.

`useEffect(() => { return () => { ... } }, [])` → Cleanup function runs when unmounting.

API

In React, an **API (Application Programming Interface)** allows your application to communicate with a server to **fetch** or **send** data.

The most common use case is fetching data from a REST API and displaying it in your React components.

1. Fetching API Data in React

React provides multiple ways to fetch data from an API:

- `fetch()` (built-in JavaScript function)
- `axios` (a popular external library)
- React Query (for advanced use cases)

Fetching Data Using **fetch()**

React component that fetches data from an API using **fetch()**.

[Click here for practical](#)

Fetch API using **async/await**

This approach makes the code cleaner and easier to read.

[Click here for practical](#)

Axios Vs Fetch

Axios: More concise and user-friendly.

Axios automatically transforms the response data into JSON.

Axios automatically rejects the request if the response status is not in the **2xx** range.

Axios automatically sets headers like **Content-Type: application/json** for **POST** requests.

Fetch only rejects requests for network errors; HTTP errors (like 404, 500) must be handled manually.

Fetch requires manually calling **.json()** on the response.

Fetch: Requires more manual handling of responses and errors.

Fetch requires setting headers manually.

Axios (Easier Alternative to Fetch)

Axios is a popular library for making HTTP requests. First, install it

npm install axios

[Click here for practical](#)

Making Post Request :

To send data to an API, use the POST method.

[Click here for practical](#)

React Query (Advanced API Handling)

React Query simplifies API calls, caching, and state management.

`npm install @tanstack/react-query`

React Query is a powerful data-fetching and state-management library for React applications. It simplifies handling server-state (API requests) with features like caching, background updates, pagination, automatic retries, and refetching.

Why Use React Query?

- Eliminates manual state management for API calls.
- Caches API responses for better performance.
- Handles background updates automatically.
- Supports pagination, infinite scrolling, and real-time updates

3) useContext – Use Context API

Avoid prop drilling by using useContext to share data globally.

The useContext hook in React is used to share state and values globally across components without the need to pass props manually (prop drilling).

It makes state management simpler in small to medium-sized applications.

Why Use **useContext**?

Normally, to pass data from a parent to a deeply nested child component, you'd need to pass props at every level (a process known as prop drilling):

Step 1 : Create a Context

create a Context using `React.createContext()`

Syntax :

```
const UserContext = createContext(null); // Default value can be null or any object
```

[Click here for practical](#)

Step 2 : Provide Context at a Higher Level

Wrap the entire component tree (or a part of it) with the Provider, passing the value.

Syntax :

```
const user = { name: "Anjali Patel", subject: "React js" }; // record
```

```
return (  
  <UserContext.Provider value={user}>  
    <ChildComponent />  
  </UserContext.Provider>  
);
```

[Click here for practical](#)

Step 3: Consume Context using useContext

Now, instead of prop drilling, use useContext inside a component to access the shared state.

Syntax :

```
import useContext from './UserContext';  
const ChildComponent = () => {  
  const user = useContext(UserContext); // Get value from Context  
  
  return <h2>Welcome, {user.name}! Age: {user.subject}</h2>;  
};
```

[Click here for practical](#)

Final Practical : [Click here](#)

Theme Change using Global State Management

- 1) Create Context : [Click here](#)
- 2) Provide context : [Click here](#)
- 3) useContext : [Click here](#)

4) useRef

`useRef` is a built-in React Hook that provides a way to persist a mutable value across renders without causing re-renders when the value changes.

It is commonly used for accessing DOM elements directly and maintaining values across renders without triggering component re-renders.

Syntax

```
import { useRef } from "react";  
  
const ref = useRef(initialValue);
```


Common Use Cases of **useRef**

1. Accessing DOM Elements

useRef is frequently used to get a reference to a DOM element and perform operations on it.

[Click here for practical](#)

2. Storing Mutable Values Without Causing Re-Renders

Unlike **useState**, changing **useRef().current** does not cause a component to re-render.

[Click here for practical](#)

5) useMemo

`useMemo` is a built-in React Hook that helps optimize performance by **memoizing** the result of a function so that it does not need to be recalculated on every render.

It is particularly useful for expensive computations and preventing unnecessary re-renders of components.

Syntax

```
import { useMemo } from "react";
```

```
const memoizedValue = useMemo(() => computeExpensiveValue(dep1,  
dep2), [dep1, dep2]);
```

Why Use **useMemo**?

- Avoid expensive recalculations
- Prevent unnecessary re-renders of child components
- Optimize performance in large applications

Common Use Cases of **useMemo**

1. Optimizing Expensive Computations

If a function performs a **costly calculation**, **useMemo** ensures it runs **only when necessary**.

[Click here for practical](#)

2. Memoizing Filtered or Sorted Lists

If a component displays a large list, filtering or sorting it on every render can be inefficient.

[Click here for practical](#)

5) useCallback

What is **useCallback**?

useCallback is a React Hook that **returns a memoized version of a callback function**, preventing unnecessary re-creation of the function on every render. It is useful when passing functions as props to child components or using them in dependencies of other Hooks like **useEffect**

Syntax

```
const memoizedCallback = useCallback(  
  () => {  
    // Function logic here  
  },  
  [dependencies]  
);
```

The first argument is the function to be memoized.

The second argument is the dependency array `[dependencies]`.

If any dependency changes, `useCallback` will return a new function reference.

Otherwise, it returns the **same function reference** from the previous render.

Why is **useCallback** Needed?

Every time a component re-renders, all functions inside it are recreated. This can cause unnecessary renders in child components that receive these functions as props

[Click here for practical](#)

[Click here for child Component](#)

Issue :

The handleClick function is re-created every time the parent re-renders.

Even though ChildComponent is wrapped in React.memo, it still re-renders because handleClick is a new function on every render.

Solution: Using **useCallback**

We can **memoize** the function using **useCallback**, so it doesn't get re-created unnecessarily.

[Click here for practical](#)

Now, handleClick only gets created once and does not change unless its dependencies change.

The child component (ChildComponent) does NOT re-render when ParentComponent updates unless necessary.

useCallback with useEffect

When using a function inside `useEffect`, if it's re-created on every render, it can **cause unnecessary re-runs** of the effect.

Problem: Infinite Loop in `useEffect`

[Click here for practical](#)

issue

The function `fetchData` gets re-created on every render.

Since `useEffect` depends on `fetchData`, it re-runs on every render, causing an infinite loop

Solution: Using `useCallback` to Stabilize the Function

[Click Here for practical](#)

`fetchData` is **memoized** using `useCallback`, so it does not get re-created on every render.

`useEffect` **only runs once** (unless dependencies change), preventing an **infinite loop**.

Custom Hooks in React.js

A custom hook in React is a reusable function that encapsulates logic using React hooks (useState, useEffect, useRef, etc.). It helps reduce code duplication and improves code organization.

Why Use Custom Hooks?

Reusability – Write logic once, use it in multiple components.

Readability – Makes components cleaner by separating concerns.

Maintainability – Easier to debug and update logic in one place.

Performance – Prevents unnecessary re-renders by isolating logic.

[Click here for custom hook](#)

[Click here for implementation of Custom Hook](#)

React Hook Form

React Hook Form (RHF) is a lightweight and efficient form management library for React. It **improves performance** by reducing unnecessary re-renders and keeping form state minimal.

Lightweight – Small bundle size (~9KB gzipped)

Faster Performance – Reduces unnecessary re-renders

Easy Integration – Works well with UI libraries (Material-UI, Ant Design, etc.)

Built-in Validation – Uses native HTML validation and custom validation

#npm install react-hook-form

[Click here for practical](#)



TOPSTECHNOLOGIES

Training | Outsourcing | Placement | Study Abroad

CRUD Operations Using Array and Object

[Click here for practical](#)

What is LocalStorage?

LocalStorage is a web API provided by the browser that allows developers to store key-value pairs in a user's browser persistently (i.e., the data remains even after the page is reloaded or the browser is closed). It is useful for caching user preferences, authentication tokens, or temporary app data.

Key Characteristics:

- Data persists even after a page refresh or browser restart.
- Stores data in key-value format (both keys and values are strings).
- 5MB limit per domain (varies by browser).
- Synchronous API (can block execution if used in large amounts).

LocalStorage provides three main methods:

`localStorage.setItem(key, value)` → Stores data

`localStorage.getItem(key)` → Retrieves data

`localStorage.removeItem(key)` → Removes a specific item

`localStorage.clear()` → Clears all data

[Click here for practical](#)

Module-11

React -Advance React- Styling , Routing

React bootstrap ;

React Bootstrap is a **React-based** implementation of the popular **Bootstrap** framework. It replaces traditional Bootstrap's **jQuery dependencies** with React components, making it more efficient and compatible with React applications.

Why Use React Bootstrap?

Fully Compatible with React – Uses React components instead of jQuery.

Pre-built UI Components – Buttons, forms, modals, and more.

Responsive & Mobile-friendly – Uses Bootstrap's grid system.

Customizable – Can be overridden with CSS or Sass.

Lightweight – Only imports required components

Installation

```
#npm install react-bootstrap bootstrap
```

Import Bootstrap CSS

```
import "bootstrap/dist/css/bootstrap.min.css";
```

[Click here for practical](#)

What is Tailwind CSS?

Tailwind CSS is a utility-first CSS framework that allows you to build modern and responsive UI without writing custom CSS. Instead of predefined components (like Bootstrap), Tailwind provides low-level utility classes that let you design directly in your HTML/JSX.

Why Use Tailwind CSS in React?

Utility-First – No need to write custom CSS files.

Highly Customizable – Tailwind can be extended via `tailwind.config.js`.

Faster Development – Directly use classes inside JSX.

Responsive Design Built-in – Supports breakpoints (sm, md, lg, etc.).

Optimized with PurgeCSS – Removes unused CSS in production for a smaller bundle size.

Installation

```
#npm install -D tailwindcss postcss autoprefixer
```

Initialize Tailwind

Run

```
#npx tailwindcss init -p
```

This will generate:

- `tailwind.config.js` (Tailwind configuration)
- `postcss.config.js` (PostCSS configuration)

Configure Tailwind to Remove Unused CSS

Edit `tailwind.config.js` to include your React file paths:

```
module.exports = {  
  content: [  
    "./src/**/*.{js,jsx,ts,tsx}", // Include all React files  
  ],  
  theme: {  
    extend: {}, // Extend if needed  
  },  
  plugins: [],  
};
```

Import Tailwind in Your CSS

In `src/index.css` or `src/globals.css`, add css

@tailwind base;

@tailwind components;

@tailwind utilities;

[Click here for practical](#)

What is React Router?

React Router is a popular **routing library for React** that allows you to create **single-page applications (SPA)** with multiple views **without refreshing the page**.

Installing React Router

To use React Router, install it via npm :

```
#npm install react-router-dom
```


React Router Components Overview

BrowserRouter – Wraps your app for routing.

Routes – Contains all route definitions.

Route – Defines a specific path and component.

Link – Used for navigation (similar to `<a>` but without page reload).

NavLink – Works like **Link** but adds an active class when the route matches.

useNavigate – A hook for programmatic navigation.

[Click here for practical](#)

Lazy loading (performance optimization)

What is Lazy Loading?

Lazy loading is a performance optimization technique that **delays the loading of non-essential resources** until they are needed. This reduces the **initial load time** and **improves page speed**, especially in large applications.

Why Use Lazy Loading?

Faster Initial Load – Loads only essential components first.

Better Performance – Reduces unnecessary resource usage.

Optimized Bandwidth – Loads content only when required.

Improves Core Web Vitals – Enhances **Largest Contentful Paint (LCP)** and **First Input Delay (FID)** scores.

[Click here for practical](#)

Module 12

React – JSON-server and Firebase Real Time Database

What is JSON Server?

JSON Server is a **lightweight, mock REST API** that allows you to create a **fake backend** using a simple `db.json` file. It helps **simulate API calls** in development without needing a real backend.

Why Use JSON Server?

Quickly create a REST API without writing backend code.

Supports CRUD operations (`GET`, `POST`, `PUT`, `DELETE`).

Works like a real API with `fetch()` or Axios.

Ideal for prototyping and testing React apps.

1. Installation

Global installation

```
npm install -g json-server
```

OR install locally in a project

```
npm install json-server --save-dev
```

2. Create a Fake Database (**db.json**)

Create a file named **db.json** in your project's root folder and add sample data:

syntax:

```
{  
  "users": [  
    { "id": 1, "name": "Alice", "email": "alice@example.com" },  
    { "id": 2, "name": "Bob", "email": "bob@example.com" },  
    { "id": 3, "name": "Charlie", "email": "charlie@example.com" }  
  ]  
}
```

3. Start the JSON Server

```
json-server --watch db.json --port 5000
```

--watch db.json → Watches db.json for changes.

--port 5000 → Runs the server on **port 5000** (default is 3000).

http://localhost:5000/users

[Click Here for practical](#)

Firestore

Firestore is a **backend-as-a-service (BaaS)** provided by Google that helps developers build and scale apps **without managing servers**. It offers various features like **authentication, real-time databases, cloud storage, hosting, and more**.

What is Firebase?

Firebase Features in React.js

Authentication – Login with Email/Password, Google, Facebook, etc.

Firestore Database – NoSQL database for real-time and scalable apps.

Realtime Database – Live data sync for chat apps and collaborative tools.

Storage – Store and serve images, videos, and files.

Hosting – Deploy React apps with Firebase Hosting.

Website: <https://firebase.google.com>

Installation :

```
#npm install firebase
```

Configure Firebase in React : Create a new file `firebaseConfig.js` and paste your Firebase SDK

syntax :

```
import { initializeApp } from "firebase/app";

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT_ID.firebaseio.com",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_PROJECT_ID.appspot.com",
  messagingSenderId: "YOUR_SENDER_ID",
  appId: "YOUR_APP_ID"
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);
export default app;
```

[Click here for practical](#)

Module-13

React - Applying Redux

Redux in React

Redux is a **state management library** used in React applications to manage global state efficiently. It helps keep track of application state **across components** and makes it easier to debug and scale apps

Redux makes state management **organized and scalable**.

It's useful for **large applications** where state sharing is essential.

What is Redux?

Redux follows the "**Single Source of Truth**" principle, where the entire state of the app is stored in a **single object (store)**.

Why Use Redux?

Centralized State – Manage global state in one place.

Predictable – State updates are controlled via actions and reducers.

Easier Debugging – Use Redux DevTools to track state changes.

Efficient – Optimized state updates prevent unnecessary re-renders.

Official Redux Website: <https://redux.js.org/>

Setting Up Redux in a React App

Step 1: Install Redux and React-Redux

Run the following command in your React project:

```
#npm install redux react-redux @reduxjs/toolkit
```

redux → Core Redux library

react-redux → Connects Redux with React

@reduxjs/toolkit → Simplifies Redux setup

Redux Async Actions (Thunk Middleware)

Redux Thunk allows you to make **API calls inside Redux actions**.

Step 1: Install Redux Thunk

```
npm install redux-thunk
```


Key Concepts of Redux

Store – The global state of the application.

Actions – Objects that describe what should change in the state.

Reducers – Functions that update state based on actions.

Dispatch – Sends an action to the store to trigger state updates.

Selectors – Get specific pieces of state from the store.

[Click Here for practical](#)

Module 14

Fetch Data using GraphQL

GraphQL

GraphQL is a query language for APIs and a runtime for executing queries against your data.

It allows clients to request only the data they need, improving efficiency and flexibility compared to REST APIs.

Key Features of GraphQL

Single Endpoint – Fetch all required data with one API call.

Flexible Queries – Clients specify exactly what data they need.

Strongly Typed Schema – Data is structured with a defined schema.

No Over-fetching or Under-fetching – Get only the necessary data.

Real-time Updates – Uses **subscriptions** for real-time data.

Official Website: <https://graphql.org/>

GraphQL Architecture

GraphQL consists of three main components:

Schema – Defines data structure and types.

Queries – Retrieve data.

Mutations – Modify data (Create, Update, Delete).

GraphQL Data Types

GraphQL provides **built-in scalar types** and allows the creation of **custom object types**.

Scalar Types (Basic Data Types)

These are primitive types that hold single values (like numbers, strings, and booleans). The most commonly used scalar types are:

- **Int** → Represents a 32-bit integer (e.g., 5, -100, 42).
- **Float** → Represents a decimal or floating-point number (e.g., 3.14, 99.99).
- **String** → Represents a text value (e.g., "Hello GraphQL").
- **Boolean** → Represents true or false values.
- **ID** → A unique identifier, often used for database records (e.g., "b1a2c3d4").

Object Types (Custom Types)

While scalar types define basic values, **object types** define structured data models with multiple fields.

For example, a `User` type can have fields like `id`, `name`, and `email`, where `id` is an **ID**, `name` is a **String**, and `email` is a **String**.

```
type User {  
  id: ID!  
  name: String!  
  email: String!  
  age: Int  
}
```

Here:

- `id`, `name`, and `email` are **mandatory fields** (because of `!`).
- `age` is **optional**.

GraphQL Modifiers

Modifiers help define **whether a field is required or if it should be an array**.

Non-Null Modifier (!)

By default, GraphQL fields can return **null**. Adding **!** ensures a field **must** have a value.

```
type Product {  
  id: ID!  
  name: String!  
  price: Float  
}
```

- **id** and **name** **must** always have values.
- **price** **can be null**.

List Modifier ([])

GraphQL allows defining **lists (arrays)** of data.

```
type Query {  
  books: [Book]  
}
```

Here, **books** returns an **array of Book objects**.

Modifiers can be combined:

- **[String]** → A **list** of strings (can be empty or null).
- **[String]!** → A **required list** (list itself cannot be **null**, but can be empty).
- **[String!]!** → A **required list** where **every element** is **non-null**.

GraphQL Schema

The **schema** defines the structure of data, the available queries, and mutations.

♦ Defining a Schema

A GraphQL schema consists of:

1. **Object Types** → Defines structured data models.
2. **Query Type** → Defines how clients fetch data.
3. **Mutation Type** → Defines how clients modify data.

[Click here for practical](#)

Setting Up a GraphQL Server

Step 1: Install Dependencies

Create a new Node.js project and install GraphQL and Apollo Server:

```
#npm init -y
```

```
#npm install express graphql express-graphql
```

[Click here for practical](#)

Apollo Client in React

Apollo Client is a **popular GraphQL client** for fetching data from a GraphQL API in React.

Install Apollo Client in React

Create a React project and install Apollo Client:

```
#npx create-react-app graphql-client
```

```
#cd graphql-client
```

```
#npm install @apollo/client graphql
```

[Click here for practical](#)



TOPSTECHNOLOGIES

Training | Outsourcing | Placement | Study Abroad

Module 15

Next JS

Introduction to Next.js

Next.js is a **React framework** that enables server-side rendering (SSR), static site generation (SSG), and many other features to build high-performance web applications.

It is developed by **Vercel** and is widely used for building **fast, SEO-friendly, and scalable** applications.

Why Use Next.js?

Next.js extends React with **powerful features** like:

Server-Side Rendering (SSR)

- Renders pages **on the server** and sends ready HTML to the client.
- Improves SEO and **faster page loads** compared to client-side rendering.

Static Site Generation (SSG)

- Generates **HTML at build time**.
- Perfect for blogs, documentation, and landing pages.

API Routes

- Built-in support for **backend APIs** without needing a separate server.
- You can create **RESTful API endpoints** inside your Next.js app.

Automatic Code Splitting

- Next.js **only loads the JavaScript needed** for the current page.
- Improves performance and **reduces load time**.

SEO Optimization

- Unlike React (which renders on the client), Next.js supports **pre-rendering** for better SEO.

Installation & Setup

To start a Next.js project, run:

```
#npx create-next-app my-next-app
```

```
#cd my-next-app
```

```
#npm run dev
```

Then open **http://localhost:3000** in your browser.

Pages & Routing

Next.js uses a **file-based routing system**.

Create a new page inside the `/pages` folder, and it automatically becomes a route.

Example:

`/pages/about.js`

Now, you can access this page at <http://localhost:3000/about>.

[Click here for practical](#)

Module 16

Introduction

Vue.js, D3.js, Chart.js

Introduction to Vue.js

Vue.js (or simply Vue) is a **progressive JavaScript framework** used to build user interfaces and **single-page applications (SPAs)**.

It was created by **Evan You** and is known for being **lightweight, flexible, and easy to learn**.

Vue.js is popular because of its:

Simple & Easy to Learn → Uses HTML, CSS, and JavaScript with a clean syntax.

Reactive Data Binding → Automatically updates the UI when data changes.

Component-Based Architecture → Breaks the UI into reusable components.

Virtual DOM → Improves performance by updating only the necessary parts of the UI.

Two-Way Data Binding → Easily syncs data between the UI and the model.

Lightweight & Fast → Smaller size compared to React or Angular.

Installing & Setting Up Vue.js

You can include Vue.js **directly** in an HTML file using a CDN:

```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
```

Or, install it using **npm**:

```
npm create vue@latest  
cd my-vue-app  
npm install  
npm run dev
```

[Click here for practical](#)

React.js vs Vue.js

React.js and Vue.js are two of the most popular JavaScript frameworks/libraries for building user interfaces.

Both have their strengths and weaknesses, making them suitable for different types of projects.

React.js

- Developed by **Facebook (Meta)** in 2013.
- A **JavaScript library** (not a full framework) used for building **component-based UIs**.
- Uses a **Virtual DOM** for fast updates.
- Requires **additional libraries** for routing (React Router) and state management (Redux, Zustand, etc.).

Vue.js

- Developed by **Evan You** in 2014.
- A **progressive framework** for building user interfaces.
- Has built-in features like **state management (Vuex, Pinia)** and **routing (Vue Router)**.
- Uses a **Virtual DOM** like React but provides better reactivity.



React Js	Vue Js
Moderate (JSX syntax may be unfamiliar)	Easier (uses regular HTML, CSS, and JS)
Well-documented but fragmented	Excellent and beginner-friendly
Faster for larger applications	Faster for smaller applications

React Js Syntax :

Uses **JSX (JavaScript XML)** to write UI components inside JavaScript.

```
function App() {  
  return <h1>Hello, React!</h1>;  
}  
export default App;
```

Vue.js (Template-based) Syntax :

- Uses HTML-based templates with special directives.

```
<template>  
  <h1>Hello, Vue!</h1>  
</template>  
<script>  
export default {  
  data() {  
    return { message: "Hello, Vue!" };  
  }  
};  
</script>
```

When to Use React vs Vue?

Use React.js if:

- You are working on a **large-scale application** (e.g., Facebook, Instagram).
- You need a **strong job market** with many opportunities.
- You prefer **JSX and JavaScript flexibility** over templates.
- You don't mind **using third-party libraries** for extra features.

Use Vue.js if:

- You want a **faster development process** with built-in tools.
- You prefer an **easy-to-learn** framework with great documentation.
- You are building a **small-to-medium app** with minimal setup.
- You want **better state management and reactivity** out of the box.

What is D3.js?

D3.js (Data-Driven Documents) is a powerful JavaScript library for creating dynamic, interactive, and data-driven visualizations in web browsers. It uses SVG, HTML, and CSS to render charts, graphs, and other visual elements.

Developed by: Mike Bostock

Initial Release: 2011

Main Purpose: Data visualization and manipulating the DOM based on data.

Used For: Bar charts, line charts, scatter plots, maps, animations, and more!

Why Use D3.js?

Highly Flexible – No predefined chart types; you build exactly what you need.

Data-Driven – Dynamically binds data to DOM elements.

Works with Large Datasets – Supports real-time updates and animations.

Built on Web Standards – Uses SVG, Canvas, HTML, and CSS.

Supports Interactivity – Zooming, panning, transitions, and animations.

Installing D3.js

Using CDN (Quick Setup)

```
<script src="https://d3js.org/d3.v7.min.js"></script>
```

Using npm (For Larger Projects)

```
npm install d3
```

Then import it in your JavaScript file:

```
import * as d3 from "d3";
```

Types of Charts we Can Build with D3.js

Bar Charts

Line Charts

Pie & Donut Charts

Scatter Plots

Heatmaps

Tree Maps & Hierarchical Charts

Network Graphs & Force-Directed Graphs

When to Use D3.js?

Great for:

- **Custom Data Visualizations** that libraries like Chart.js can't handle.
- **Interactive Dashboards** with real-time updates.
- **Big Data Visualizations** for analytics and reporting.

Not Ideal for:

- Simple charts (Use Chart.js or Google Charts for easier solutions).
- Projects requiring fast development with minimal customization.

[Click here for practical](#)

Types of Charts we Can Build with D3.js

Bar Charts

Line Charts

Pie & Donut Charts

Scatter Plots

Heatmaps

Tree Maps & Hierarchical Charts

Network Graphs & Force-Directed Graphs

Introduction to Chart.js

Chart.js is a simple, flexible, and lightweight JavaScript library used for creating beautiful and interactive charts in web applications. It is based on the HTML5 Canvas API and supports multiple chart types with smooth animations.

Developed by: Nick Downie

Initial Release: 2013

File Size: Lightweight (~60KB minified)

Main Purpose: Creating charts, graphs, and data visualizations easily.

Why Use Chart.js?

Chart.js is widely used because it is:

Easy to Use – Simple API for creating charts with minimal code.

Lightweight – Small file size (~60KB minified) compared to libraries like D3.js.

Highly Customizable – Supports colors, labels, legends, tooltips, and more.

Responsive & Interactive – Charts adjust to different screen sizes.

Supports Multiple Chart Types – Bar, Line, Pie, Radar, Doughnut, Bubble, etc.

Works with JavaScript Frameworks – Compatible with React, Vue, Angular, etc.

Installing Chart.js Using CDN (Quick Setup)

```
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

Using npm (For Larger Projects)

```
npm install chart.js
```

Then import it in your JavaScript file:

```
import Chart from "chart.js/auto";
```

[Click here for practical](#)

Advantages of Chart

Visually Appealing & Customizable

- Supports various chart types (bar, line, pie, radar, etc.).
- Customizable colors, tooltips, legends, and animations.

Lightweight & Fast

- Small file size compared to other charting libraries.
- Uses the HTML5 Canvas element, making it efficient for rendering.

Easy to Use & Integrate

- Simple API with clear documentation.
- Works well with React, Vue, Angular, and other frameworks.

Responsive & Interactive

- Automatically adjusts to different screen sizes.
- Supports hover effects, tooltips, and animations.

Supports Real-time Data Updates

- Can update and animate charts dynamically.
- Useful for live dashboards and monitoring tools.

Open-source & Community Supported

- Free to use under the MIT license.
- Active community for support and improvements.